Desenvolvimento de Sistemas Web II

Kotlin para quem vem do Java

DIM0547 — Turma 01 · Sprint 1 · apoio

Prof. Fernando · UFRN · 2026.2

Aviso: prova adiada para 11/11

	Antes	Agora
Prova escrita	09/11 (segunda)	**11/11 (quarta)**, em laboratório, no horário da aula
09/11 (segunda)	presencial: prova	🔵 online: acompanhamento de projetos
11/11 (quarta)	online: acompanhamento	🟢 presencial: **prova escrita**
Prova de reposição	02/12	02/12 (sem mudança)

- Vale no lugar do calendário dos slides 05; o conteúdo e as regras da prova são os mesmos
- Entrega da Sprint 1: **16/10 (sexta), 23:59**

O que esta aula responde

Um trecho do `Rotas.kt` do exemplo de Tarefas, e as perguntas que ele costuma levantar:

```
fun Application.rotas() {                                   // 1
    val repositorio by inject<RepositorioDeTarefas>()       // 2

    routing {                                               // 3
        get("/tarefas/{id}") {                              // 4
            val id = call.parameters["id"]?.toIntOrNull()   // 5
                ?: throw BadRequestException("O id deve ser um número inteiro")
            call.respond(repositorio.buscar(id) ?: throw NotFoundException("..."))
        }
    }
}
```

1. Por que há um tipo e um ponto antes do nome da função? → *função de extensão*
2. O que o `by` faz? → *delegação*
3. `routing` é palavra da linguagem? Cadê os parênteses? → *lambda no fim da chamada*
4. De onde vem `call`, se ninguém o declarou? → *lambda com receptor*
5. O que são `?.` e `?:` → *nulidade no tipo*

Relembrando: o que muda na escrita

Java

```java
final String titulo = "Estudar";
int total = 0;
total = total + 1;

String situacao(Tarefa t) {
    return t.feita() ? "feita" : "pendente";
}

var t = new Tarefa(1, titulo, false);
System.out.println("Tarefa " + t.id());
```

Kotlin

```kotlin
val titulo = "Estudar"     // não muda
var total = 0              // muda
total = total + 1

fun situacao(t: Tarefa): String =
    if (t.feita) "feita" else "pendente"

val t = Tarefa(1, titulo)
println("Tarefa ${t.id}")
```

- O tipo vem depois do nome (`t: Tarefa`) e pode ser omitido quando o compilador o deduz
- Sem `new`, sem ponto e vírgula; `Unit` faz o papel do `void`
- Função de uma expressão: `=` no lugar de `{ return ... }`

📖 **Ref.** Kotlin — Basic syntax

`if` e `when` devolvem valor

```kotlin
// if no lugar do operador ternário
val situacao = if (tarefa.feita) "feita" else "pendente"

// when: o valor do ramo escolhido é o valor da função (Tarefa.kt, Web II)
fun violacoes(): List<String> = when {
    titulo.isBlank()    -> listOf("titulo: não pode ficar em branco")
    titulo.length > 200 -> listOf("titulo: no máximo 200 caracteres")
    else                -> emptyList()
}
```

- Em Java, `if` é comando: a variável é declarada antes e atribuída em cada ramo
- Em Kotlin, `if`, `when` e `try` são expressões: o resultado vai direto para o `val`
- Por isso a linguagem não tem o operador `? :`

📖 **Ref.** Kotlin — Conditions and loops · ▶ Demonstração 1

Nulidade faz parte do tipo

Escrita	Significado	Em Java
`Tarefa`	nunca é `null`	sem equivalente; depende de convenção
`Tarefa?`	pode ser `null`	toda referência
`a?.b`	`b` se `a` não for `null`; senão `null`	`a != null ? a.b : null`
`a ?: c`	`a` se não for `null`; senão `c`	`a != null ? a : c`
`a!!`	"garanto que não é `null`"; falha em execução se for	uso direto, com risco de `NullPointerException`

```
// Rotas.kt (Web II): cada ?: decide o que fazer quando o valor falta
val id = call.parameters["id"]?.toIntOrNull()
    ?: throw BadRequestException("O id deve ser um número inteiro")
val tarefa = repositorio.buscar(id)
    ?: throw NotFoundException("A tarefa $id não existe")
call.respond(tarefa)              // aqui `tarefa` já é Tarefa, não Tarefa?
```

📖 **Ref.** Kotlin — Null safety · ▶ Demonstração 1

Classes: o construtor declara as propriedades

Java

```java
public record Tarefa(
    int id, String titulo, boolean feita) {

  public Tarefa(int id, String titulo) {
    this(id, titulo, false);
  }
}

var a = new Tarefa(1, "Estudar");
var b = new Tarefa(
    a.id(), a.titulo(), true);
```

Kotlin

```kotlin
data class Tarefa(
    val id: Int,
    val titulo: String,
    val feita: Boolean = false,
)



val a = Tarefa(1, "Estudar")
val b = a.copy(feita = true)
```

- `data class` gera `equals`, `hashCode`, `toString` e `copy`; `==` compara os valores
- Valor padrão (`= false`) substitui construtores sobrecarregados
- Argumento nomeado (`feita = true`) deixa claro o que cada valor é
- `object` declara uma instância única; `companion object` guarda o que em Java seria `static`

📖 **Ref.** Kotlin — Data classes · ▶ Demonstração 2

Entendendo: lambda

Uma lambda é um bloco de código guardado num valor. O tipo dela se escreve `(Entrada) -> Saída`.

```
val feita: (Tarefa) -> Boolean = { tarefa: Tarefa -> tarefa.feita }   // completa
val feita: (Tarefa) -> Boolean = { tarefa -> tarefa.feita }          // tipo do parâmetro deduzido
val feita: (Tarefa) -> Boolean = { it.feita }                        // um parâmetro só: `it`
```

Tipo em Kotlin	Interface em Java	Exemplo no curso
`() -> Unit`	`Runnable`	`onAdicionar`, `onVoltar`
`(Int) -> Unit`	`Consumer<Integer>`	`onAlternar`, `onAbrir`
`(Tarefa) -> Boolean`	`Predicate<Tarefa>`	a condição do `filter`
`(Tarefa) -> String`	`Function<Tarefa, String>`	a transformação do `map`

- As chaves delimitam a lambda; a seta separa parâmetros e corpo
- O valor da lambda é a última expressão; não se escreve `return`

📖 **Ref.** Kotlin — Higher-order functions and lambdas · ▶ Demonstração 3

Entendendo: lambda no fim da chamada

Quando o último parâmetro é uma função, a lambda pode sair dos parênteses. A mesma chamada, em quatro formas:

```
tarefas.filter({ tarefa -> !tarefa.feita })    // 1. lambda como argumento comum
tarefas.filter() { tarefa -> !tarefa.feita }   // 2. lambda depois dos parênteses
tarefas.filter { tarefa -> !tarefa.feita }     // 3. parênteses vazios somem
tarefas.filter { !it.feita }                   // 4. com `it`
```

É a regra que explica a forma de quase todo o código do curso:

```
install(ContentNegotiation) { json() }                 // Ktor
Button(onClick = onAdicionar) { Text("Adicionar") }    // Compose
items(tarefas, key = { it.id }) { tarefa -> ... }      // duas lambdas: uma nomeada, uma no fim
```

> Ao ler `nome { ... }` ou `nome(args) { ... }`: é uma chamada de função, e o bloco é o último argumento.

📖 **Ref.** Kotlin — Passing trailing lambdas · ▶ Demonstração 3

Coleções: as operações são funções da lista

Java

```java
List<String> pendentes = tarefas.stream()
    .filter(t -> !t.feita())
    .map(t -> t.titulo())
    .toList();

List<Tarefa> depois = tarefas.stream()
    .map(t -> t.id() == id
        ? new Tarefa(t.id(), t.titulo(),
                     !t.feita())
        : t)
    .toList();
```

Kotlin

```kotlin
val pendentes = tarefas
    .filter { !it.feita }
    .map { it.titulo }


val depois = tarefas.map {
    if (it.id == id)
        it.copy(feita = !it.feita)
    else it
}
```

- `List` é somente leitura; para alterar no lugar, o tipo é `MutableList`
- "Alterar" uma lista imutável é criar outra: `map`, `filter`, `+`
- No Compose, é a reatribuição da lista nova que avisa a tela para se redesenhar

📖 **Ref.** Kotlin — Collections in Java and Kotlin · ▶ Demonstração 4

Entendendo: função de extensão

Acrescenta uma função a um tipo que já existe, sem herança e sem alterar a classe.

Java — método estático utilitário

```java
class Tarefas {
  static List<Tarefa> alternando(
      List<Tarefa> lista, int id) { ... }
}

Tarefas.alternando(lista, 2);
```

Kotlin — o tipo antes do ponto é o receptor

```kotlin
fun List<Tarefa>.alternando(id: Int) =
    map { if (it.id == id)
        it.copy(feita = !it.feita) else it }

lista.alternando(2)
```

- Dentro da função, `this` é o receptor, e pode ser omitido (`map` é `this.map`)
- Só enxerga a parte pública do tipo; é uma função comum, escolhida na compilação
- No curso: `fun Application.modulo()`, `fun Application.rotas()`, `fun List<Tarefa>.comNova(...)`

📖 **Ref.** Kotlin — Extensions · ▶ Demonstração 5

Entendendo: lambda com receptor

Junta as duas ideias: uma lambda que, por dentro, tem um `this`.

Lambda comum: `(Conexao) -> Unit`

```
configurar { conexao ->
    conexao.url = "jdbc:..."
    conexao.usuario = "tarefas"
}
```

Com receptor: `Conexao.() -> Unit`

```
configurar {
    url = "jdbc:..."          // this.url
    usuario = "tarefas"       // this.usuario
}
```

```
// Banco.kt (Web II): apply recebe uma lambda com receptor e devolve o próprio objeto
HikariConfig().apply {
    jdbcUrl = config.url          // this.jdbcUrl
    maximumPoolSize = 5
}
```

> Dentro de um bloco, pergunte: quem é o `this` aqui? A resposta está no tipo do parâmetro: `Tipo.() -> Unit`.

📖 **Ref.** Kotlin — Function literals with receiver · ▶ Demonstração 6

Lendo o código do Ktor com essas três peças

```
fun Application.rotas() {                          // extensão de Application
    val repositorio by inject<RepositorioDeTarefas>()

    routing {                                      // lambda no fim; this: Routing
        route("/tarefas") {                        // this: Route
            get("/{id}") {                         // this: RoutingContext (é dele o `call`)
                val id = call.parameters["id"]?.toIntOrNull() ?: throw BadRequestException("...")
                call.respond(repositorio.buscar(id) ?: throw NotFoundException("..."))
            }
        }
    }
}
```

O mesmo trecho, sem as abreviações da linguagem:

```
this.routing({ this.route("/tarefas", { this.get("/{id}", { this.call.respond(...) }) }) })
```

- Cada bloco é o último argumento de uma função, e cada função define o `this` do seu bloco
- As rotas são chamadas de função comuns: por isso cabem `if`, laços e funções auxiliares

📖 **Ref.** Ktor — Routing · Kotlin — Type-safe builders · ▶ Demonstração 7

Funções de escopo

Cinco funções da biblioteca padrão que executam um bloco sobre um objeto.

Função	O objeto é	Devolve	Uso típico	No curso
`let`	`it`	o resultado do bloco	fazer algo só se não for `null`: `x?.let { }`	`selecionada?.let(alternar)`
`apply`	`this`	o próprio objeto	configurar um objeto recém-criado	`HikariConfig().apply { }`
`also`	`it`	o próprio objeto	efeito extra, como um registro	—
`run`	`this`	o resultado do bloco	calcular algo a partir do objeto	—
`with(x)`	`this`	o resultado do bloco	várias chamadas sobre o mesmo objeto	—

- Para ler: primeiro veja se o objeto é `it` ou `this`; depois, o que a chamada devolve
- Para escrever: comece por `let` e `apply`, que cobrem a maior parte dos casos

📖 **Ref.** Kotlin — Scope functions · ▶ Demonstração 6

Delegação: o que o `by` faz

`by` entrega a outro objeto o trabalho de ler (e gravar) uma propriedade.

```
// Rotas.kt: a dependência é buscada no Koin no primeiro uso, e guardada
val repositorio by inject<RepositorioDeTarefas>()

repositorio.listar()          // usa-se como um RepositorioDeTarefas comum
```

```
// A mesma ideia na biblioteca padrão: calculado só no primeiro uso
val json by lazy { Json { explicitNulls = false } }
```

- Sem `by`, a variável guardaria um `Lazy<RepositorioDeTarefas>`, e cada uso seria `repositorio.value.listar()`
- Com `by`, o compilador troca cada leitura por uma chamada a `getValue()` do delegado
- `inject<RepositorioDeTarefas>()`: o tipo entre `< >` substitui o `RepositorioDeTarefas.class` do Java

📖 **Ref.** Kotlin — Delegated properties · Koin — Ktor · ▶ Demonstração 9

`suspend`: esperar sem prender a thread

```
// Tarefa.kt: a porta do repositório
interface RepositorioDeTarefas {
    suspend fun listar(): List<Tarefa>
    suspend fun buscar(id: Int): Tarefa?
    suspend fun adicionar(nova: NovaTarefa): Tarefa
}
```

- `suspend` marca uma função que pode pausar (enquanto o banco responde) e continuar depois
- Enquanto ela espera, a thread atende outras requisições
- Uma função `suspend` só pode ser chamada de outra função `suspend` ou de dentro de uma corrotina
- No Ktor, o bloco de cada rota já é `suspend`: por isso `repositorio.buscar(id)` é chamado direto
- Em Java, o caminho equivalente passa por `CompletableFuture` ou por threads virtuais

	Thread da JVM	Corrotina
Quem gerencia	o sistema operacional	a biblioteca `kotlinx.coroutines`
Custo	alguns MB cada	leve: milhares numa mesma thread

📖 **Ref.** Kotlin — Coroutines basics · ▶ Demonstração 10

Escrever de modo idiomático

Java escrito em Kotlin

```
fun descrever(tarefa: Tarefa?): String {
    var texto: String
    if (tarefa == null) {
        texto = "nenhuma tarefa"
    } else {
        if (tarefa.feita) {
            texto = tarefa.titulo + " (feita)"
        } else {
            texto = tarefa.titulo + " (pendente)"
        }
    }
    return texto
}
```

Kotlin idiomático

```
fun descrever(tarefa: Tarefa?) = when {
    tarefa == null -> "nenhuma tarefa"
    tarefa.feita -> "${tarefa.titulo} (feita)"
    else -> "${tarefa.titulo} (pendente)"
}
```

- Prefira `val`; use `var` só quando o valor muda
- Use `if` e `when` como expressão, e template de string no lugar de `+`
- Use `filter` e `map` no lugar de laço com índice; `data class` no lugar de getters escritos à mão
- Evite `!!`: prefira `?.`, `?:` ou um teste de `null`

📖 **Ref.** Kotlin — Idioms · Coding conventions · ▶ Demonstração 11

Demonstrações no Kotlin Playground

Cada link abre o código já carregado em play.kotlinlang.org, e o botão **Run** executa. Cada demonstração termina com três experimentos. Os arquivos também estão em `exemplos/kotlin-playground/`.

	Demonstração	O que mostra
1	▶ Expressões e nulidade	`if`, `when`, `?.`, `?:`
2	▶ Classes	`data class`, `copy`, `object`, `companion object`
3	▶ Lambdas	tipos de função, `it`, lambda no fim da chamada
4	▶ Coleções	`filter`, `map`, `find`, listas imutáveis
5	▶ Funções de extensão	receptor, `this`, propriedade de extensão
6	▶ Lambda com receptor	`Tipo.() -> Unit`, `apply`, `let`, `also`, `run`, `with`
7	▶ Um Ktor em miniatura	`routing { get("/tarefas") { } }` em 30 linhas
8	▶ Um Compose em miniatura	`Coluna { Texto("...") }` e eventos por lambda
9	▶ Delegação	`by`, `by lazy`, um delegado que observa mudanças
10	▶ Corrotinas	`suspend`, `async`, dez mil corrotinas
11	▶ Idiomático	o mesmo programa em estilo Java e em estilo Kotlin

Para estudar

- Leitura: `leituras/web2-s1-kotlin.md` — o conteúdo desta aula por extenso, com os trechos do exemplo de Tarefas comentados e exercícios
- Tour of Kotlin: trilha oficial com código executável e exercícios; a parte intermediária cobre extensões, funções de escopo e lambdas com receptor
- Kotlin Koans: exercícios no navegador, pensados para quem vem do Java

Fontes desta aula

- Kotlin — Comparison to Java · Lambdas · Extensions · Scope functions · Null safety · Idioms
- Kotlin — Delegated properties · Type-safe builders · Coroutines basics
- Ktor — Routing · JetBrains — Kotlin for Education

A documentação do Kotlin é publicada sob a licença Apache 2.0. Os trechos de código vêm do exemplo de Tarefas deste repositório.